

Introduction to Artificial Intelligence

Week 5

Agenda

- Uniformed Search
- Informed Search
- Backtracking
- Hill Climbers

Learning by Searching

Part I

Your Car is Lost in a Carpark



"Here ya go. I'm always losing mine in big parking lots, too."

- How would you find it?
- Walk at random
  - No guarantee you will find it
  - Could be lost forever
- Walk down each row
  - Guarantee you will find it
  - Might take a long time
- Walk to the place you think it was last, then expand your search
  - Guarantee you will find it
  - As long as your memory is good – on average it will be less than every row, worst case just as good
- Walk to the place you think it was last, hit the alarm button, then expand search
  - Guarantee you will find it
  - Alarm can be heard from a long way away, and if you press it to no effect the car is not local to you (radius of activation)

Search Space

- Many problems define a set of actions on an environment which can be seen as a search space of states
  - We are at one point in the environment with a destination, there is a list of action states which will bring us to the destination
- Each search type is a method of navigation of the space
- We will rarely want to go to the same point in the search space
  - Represents a cycle in the search, which is not useful in terms of state spaces
- Solution – most searches will avoid this by using tree structures
  - Each node in the tree is a state within the search
  - Search Trees, Game Trees

Types of Searches

Uninformed Search

- Have no sense of the problem domain
- Generally applicable in all cases

Informed Search

- Use a heuristic function developed for the domain
- Applicable in their own domain

Goal-based Agent



- Four Steps in the Agent:

1. Define Goals
   - What are the states which are considered to be satisfying the goal?
2. Problem Formulation
   - What actions are available to move towards the goal?
   - What states are available to move towards the goal?
3. Search
   - Determine the pathway of states in order to meet with the goal
4. Execute
   - Move through the series of states

Example: Painting Robot (simplified)



1. Goal
   - All show surfaces of the part are covered in 1mm of paint
2. Problem Formulation
   - A sequence of points about the part for the robot arm to move into in order to ensure a dispersal about the entire part
3. Search
   - Examine all pathways about the part, taking into account the movements on each part of the arm
   - A pathway is good as long as it covers the part with 1mm of paint
4. Execute
   - The arm moves through the sequence of positions

States Space of the Robot

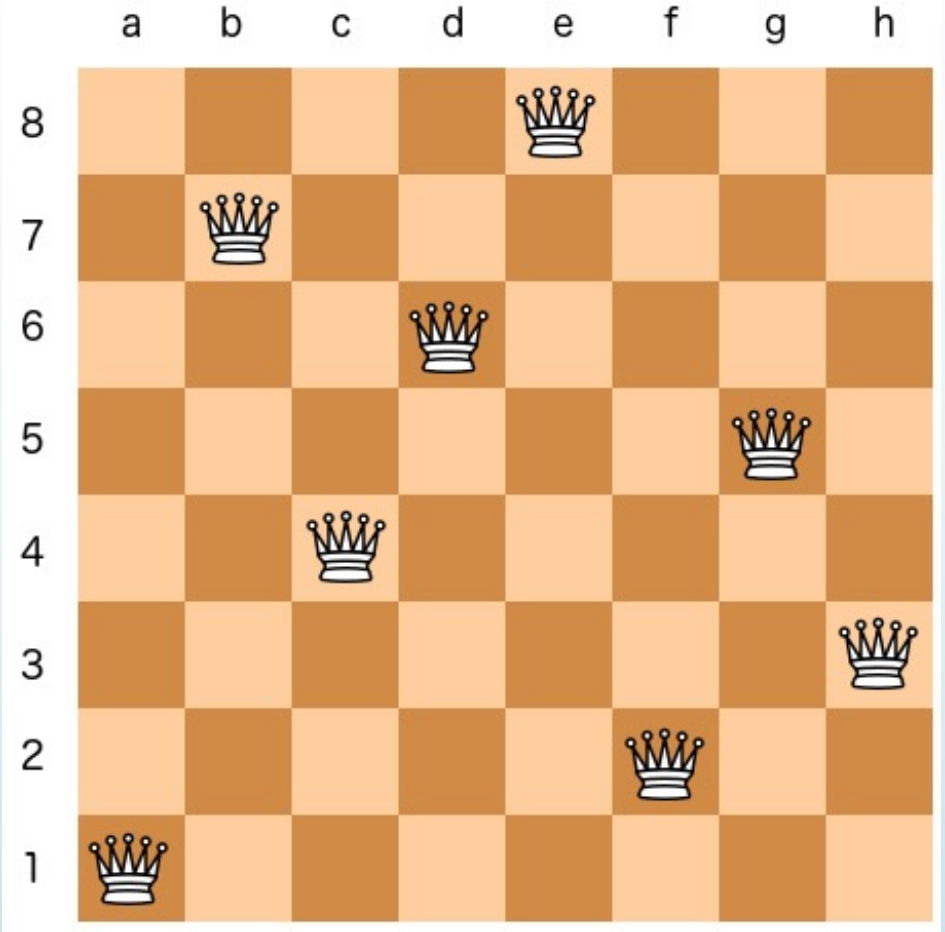
- State space is defined as the location of all moving elements on the robot
  - Relative positioning on all joints from a home position
  - Might also include the rate of spray on the nozzle – from 0 to full
- Transitions of states could be broken down into a sequence of movements in the joints on each rotational axis (hence a 6-axis robot)
  - Move(Elbow, X degrees)



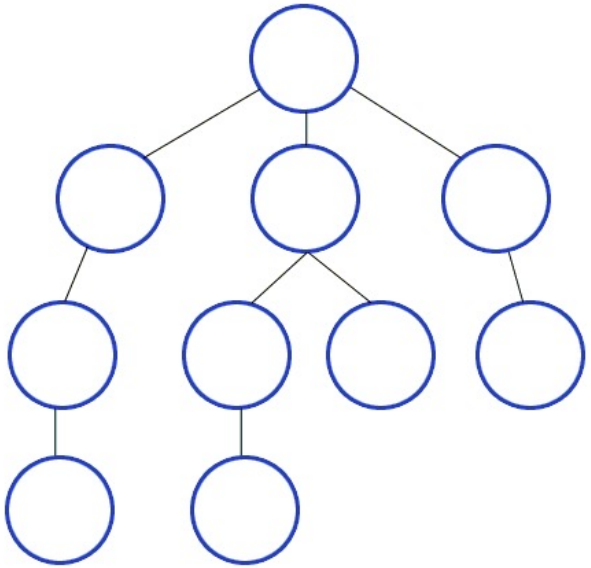
N Queens and Incremental Goals



- Place N queens onto an NxN chess board such that no queen attacks any other queen; a more complicated problem is a Costas Array
- States
  - Place a queen down
- Goal
  - No queen attacks any other queen
  - Hard sometimes to see a good pathway to the goal
- Need a measure in order to see a good pathway
  - Minimize the number of queen attacks



Tree Search



- Recursive definition of a tree search as depth-first search (DFS)
- SearchTree(State, Move, Visited list)
  - New State <- Apply Move
  - If (New State == Goal) – return TRUE
  - Else
    - If neighbouring states are empty - return FALSE
    - For each of the MOVE on neighbouring states not in Visited list
      - If SearchTree(Neighbouring states, MOVE, Visited list + State) return TRUE;

Backtracking Searches

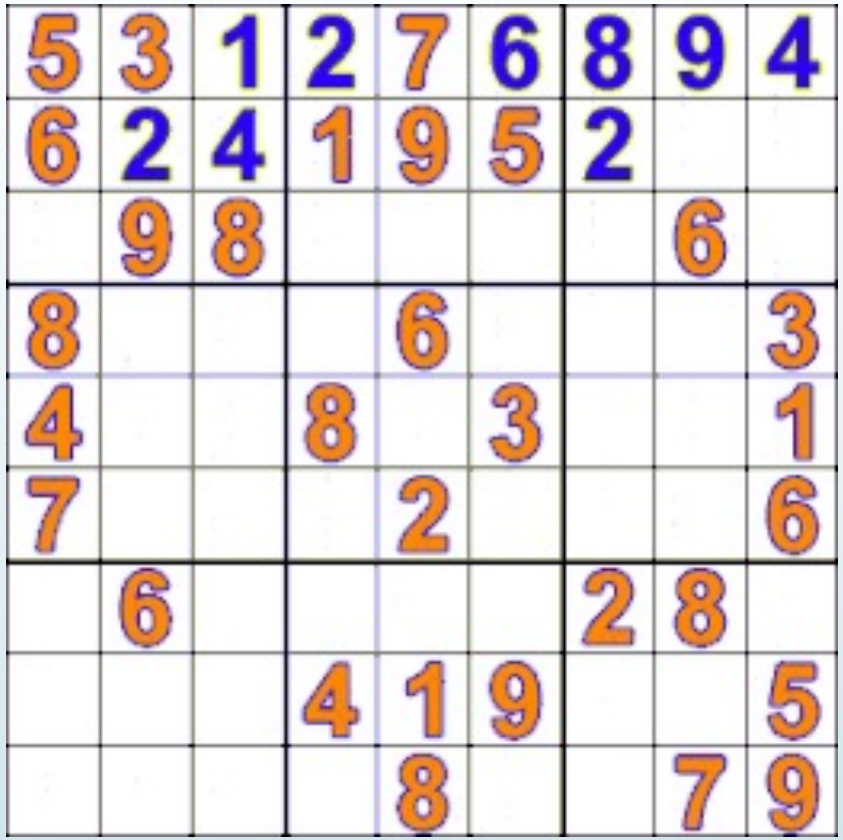
- Move towards the goal. If you reach a position which is going to be less successful – stop and move back up the tree (backtrack, retrace your steps)
- Recursive definition of a DFS of a backtrack:
- SearchTreeBacktrack(State, Move, Visited list)
  - New State <- Apply Move
  - If (New State == Goal) – return TRUE
  - If (New State causes an invalid path to goal/or costs too much) – return FALSE
  - Else
    - If neighbouring states are empty - return FALSE
    - For each of the MOVE on neighbouring states not in Visited list
      - If SearchTreeBacktrack(Neighbouring states, MOVE, Visited list + State) return TRUE;

Sudoku

	1		4		2		5	
5								6
			3		1			
7		5				4		8
2		8				5		9
			9		6			
6								2
	7		1		3		4	

Backtrack on Sudoku

- Is valid?
  - Check if we have not violated the rules
    - 1-9 in each row
    - 1-9 in each column
    - 1-9 in each box
- Choice at each empty box 1-9
- Multiple Solutions?



Heuristic Search

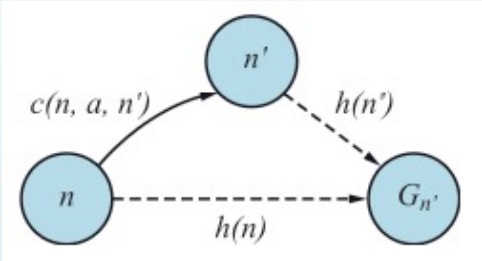
- Word *heuristic* comes from the Greek for discovery
- Heuristics use an educated guess on where to move the search next – rules-based movement about a search space
- Not necessarily deterministic – may have stochastic generation methods
- Trial and Error
  - Experimental and Experiential
  - Does not guarantee the most optimal solution
- Examples
  - *Greedy Approaches* and *Hill Climbers* are heuristics

Heuristics Admissibility

- In path-finding problems heuristics may be *admissible* or *inadmissible*
- Admissibility means that heuristics is not overestimating the solution
  - Needs to be lower than or equal to the actual cost of reaching the goal
  - An admissible heuristic is therefore optimistic
- Not each heuristic is obliged to be admissible. However, inadmissible heuristic does not guarantee the optimal solution

Heuristics Consistency

- In path-finding problems a heuristic is considered to be *consistent* if the estimated cost from one node to the successor node, added to the estimated cost from the successor node to the goal is less than or equal to the estimated cost from the current node to the goal state
- $h(n) \leq c(n, a, n') + h(n')$

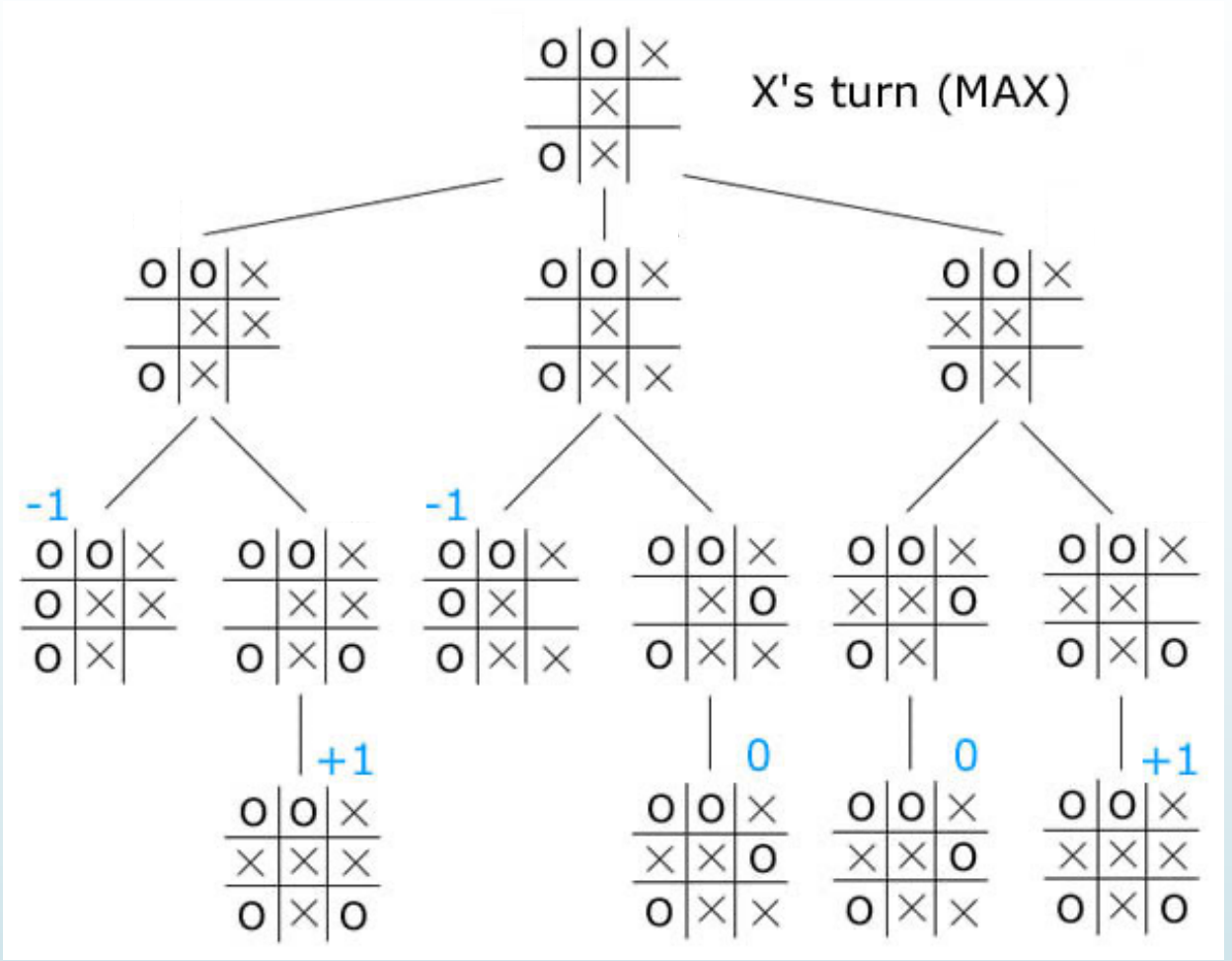


- Consistent heuristics are called *monotone* because the estimated final cost of a partial solution is monotonically non-decreasing along any path
- All consistent heuristics are admissible heuristics, however, all admissible heuristics are not necessarily consistent heuristics

Minimax

- Minimax is a kind of backtracking algorithm that is used in decision making
- Opponent's goal is to reduce winnings (MINI-Move) and player's goal is to increase winnings (MAX-Move)
- Game with a scoring system or win/lose condition which can be evaluated
  - GO
  - Chess
- Game Tree
  - Selection of plays in a turn-based game can be reduced to looking at a pathway on a tree
  - Used as part of the theoretical foundations of GAME THEORY

Minimax Example

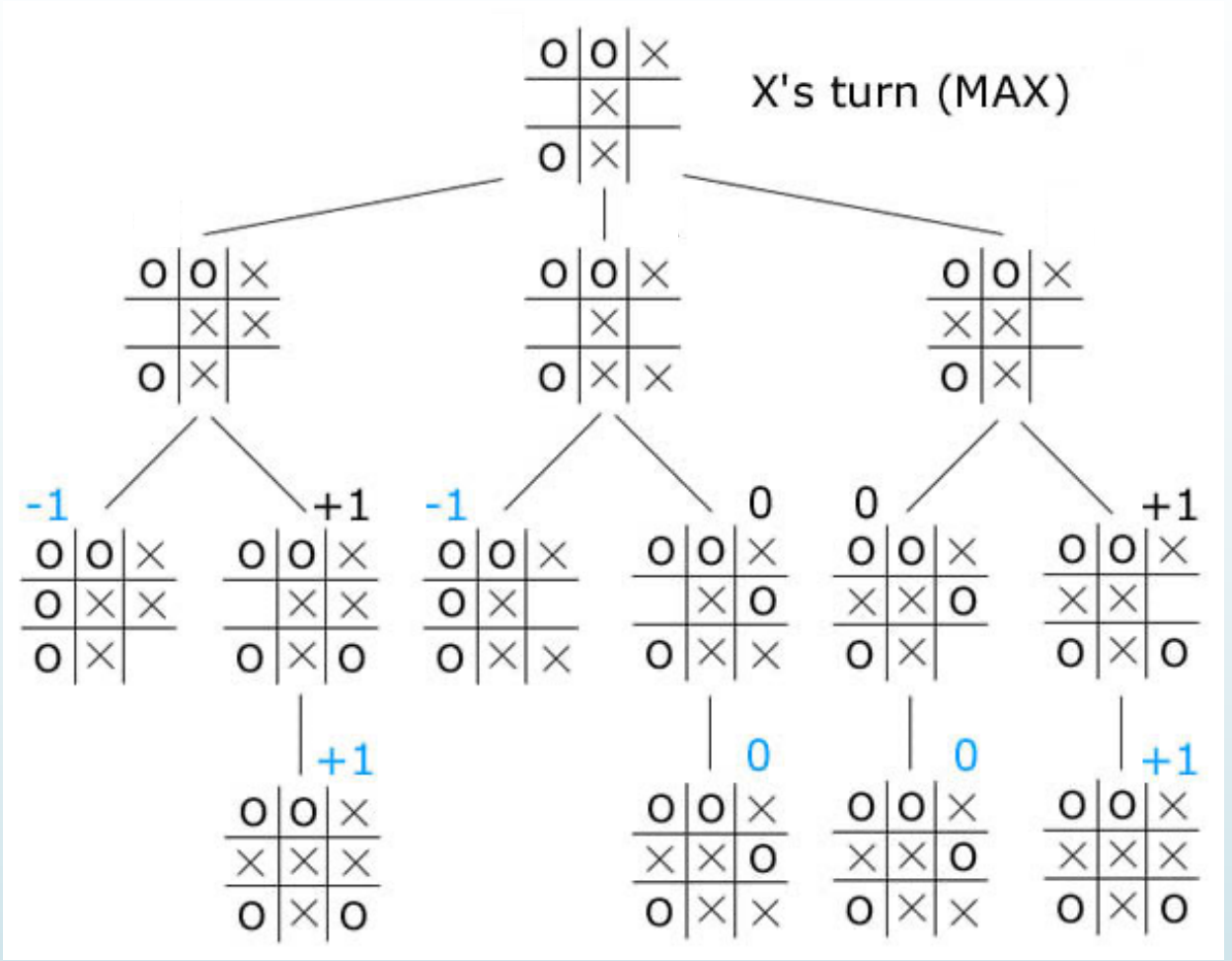


X’s turn (MAX)

0’s turn (MIN)

X’s turn (MAX)

Minimax Example

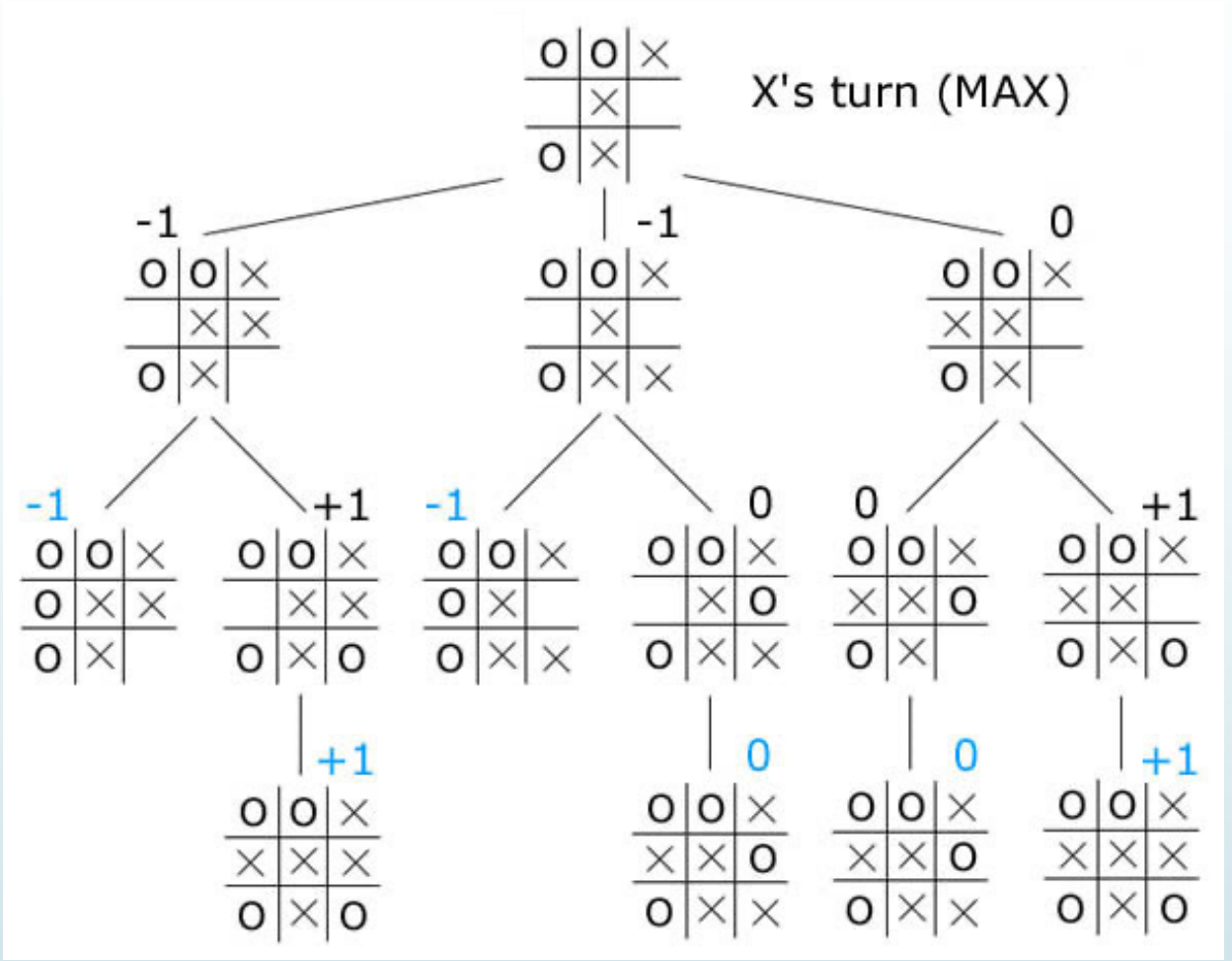


X’s turn (MAX)

0’s turn (MIN)

X’s turn (MAX)

Minimax Example

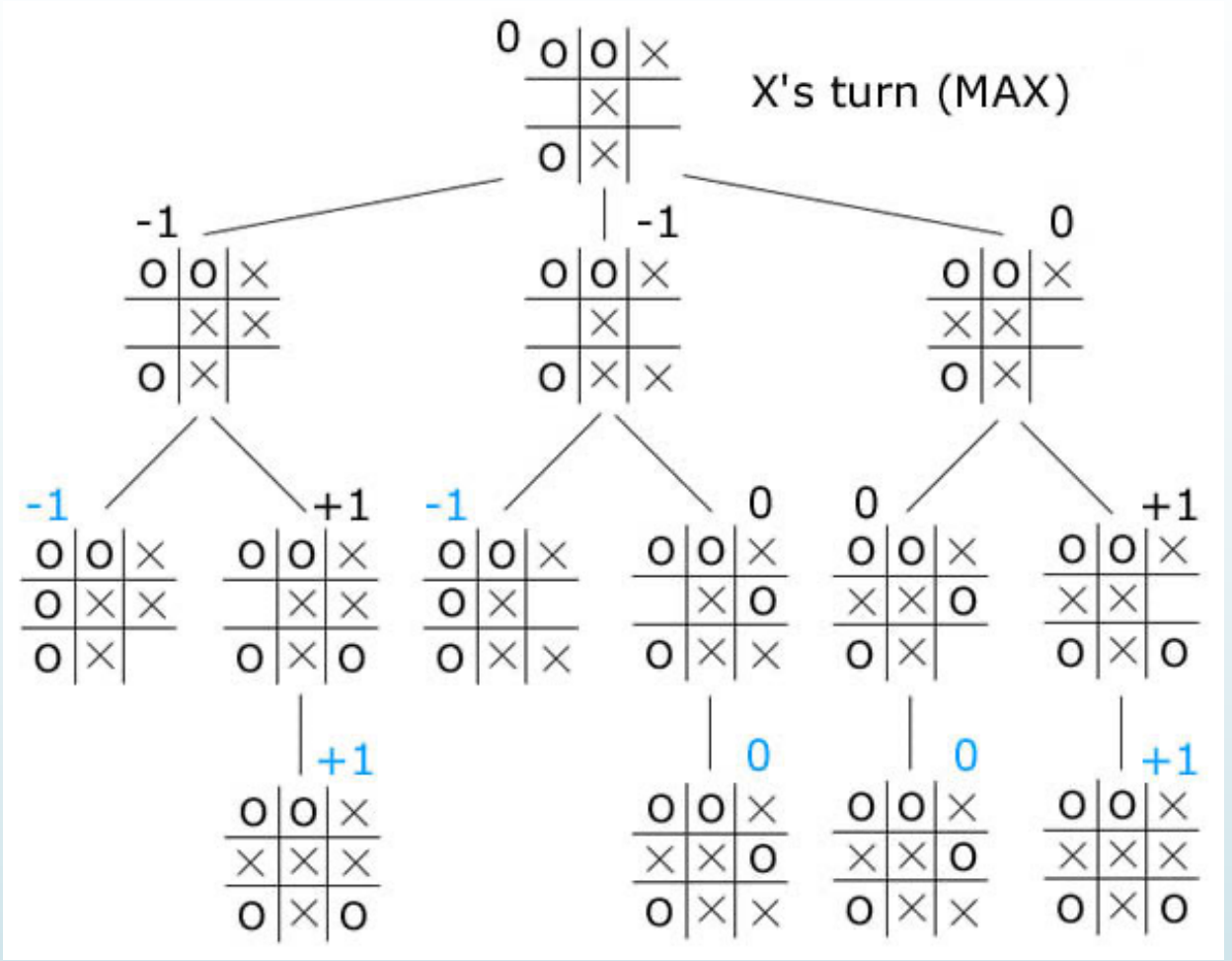


X’s turn (MAX)

0’s turn (MIN)

X’s turn (MAX)

Minimax Example



X’s turn (MAX)

0’s turn (MIN)

X’s turn (MAX)

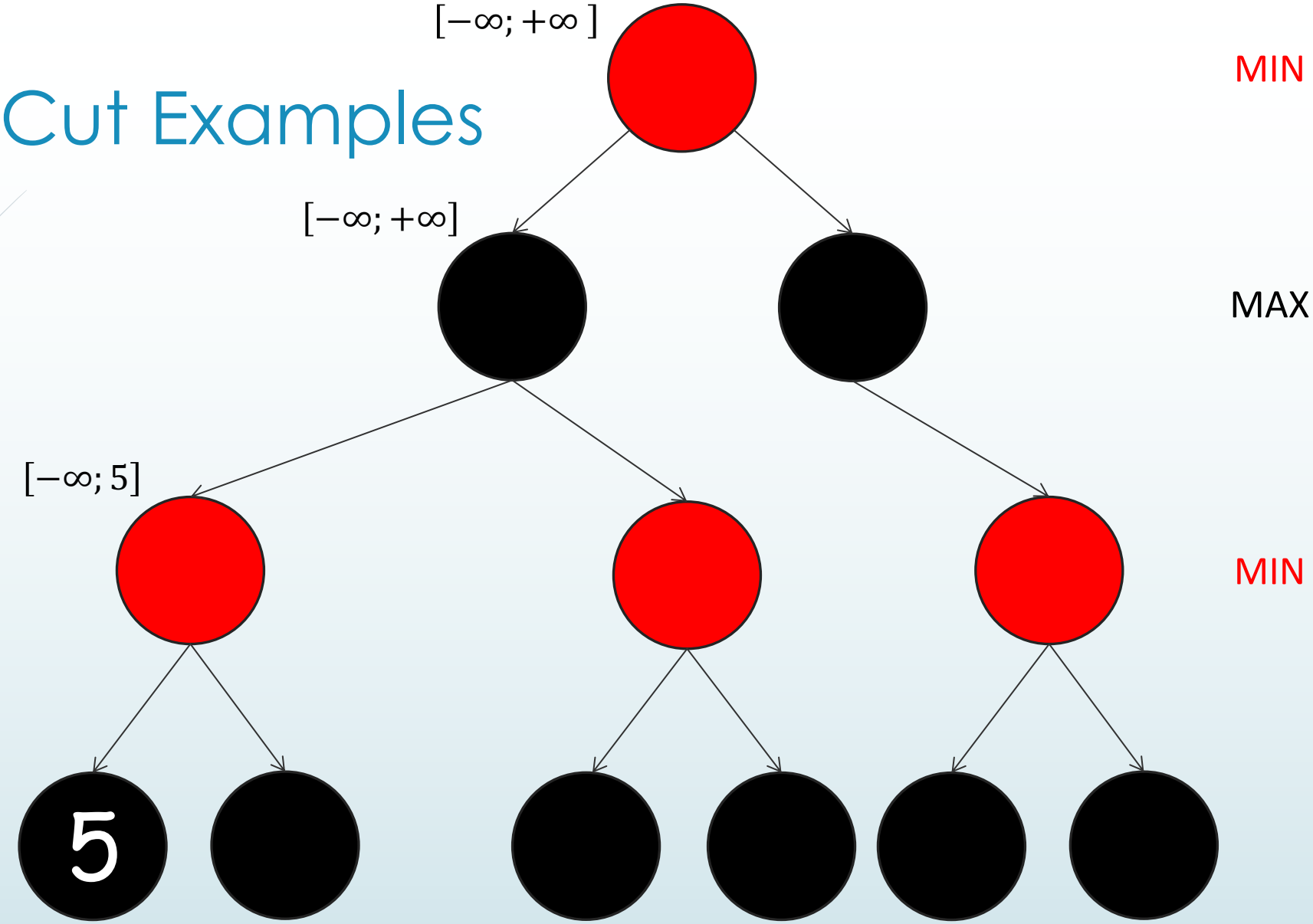
Backtracking

- Simulate the entire game
- Assume rational/perfect play from the opponent
- Theoretically, there is a perfect game play method if we search the entire game
- Solved Games
  - Tic-Tac-Toe
  - Checkers
  - Two player heads-up limit poker – Recent discovery by researchers in University of Alberta
- Unsolved
  - Chess
  - Go

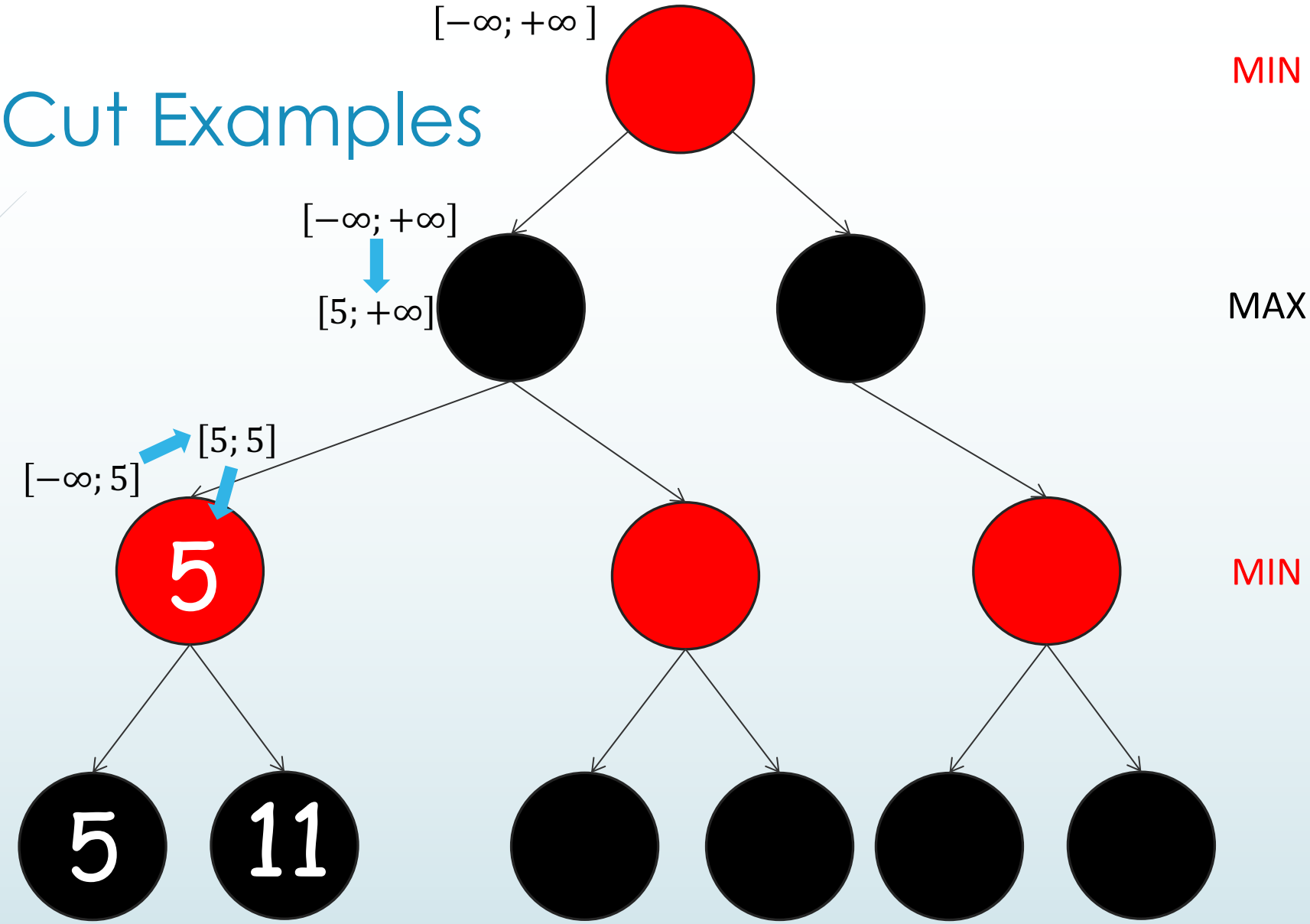
Backtracking with Alpha-Beta Pruning

- Heuristic search based on backtracking
- Used in situations where there is a value to the solution at each point to allow for an evaluation
- Applied for Game Trees
- Alpha Cut
  - Maximum lower bound (Max Play - Black)
- Beta Cut
  - Minimum upper bound (Min Play - Red)

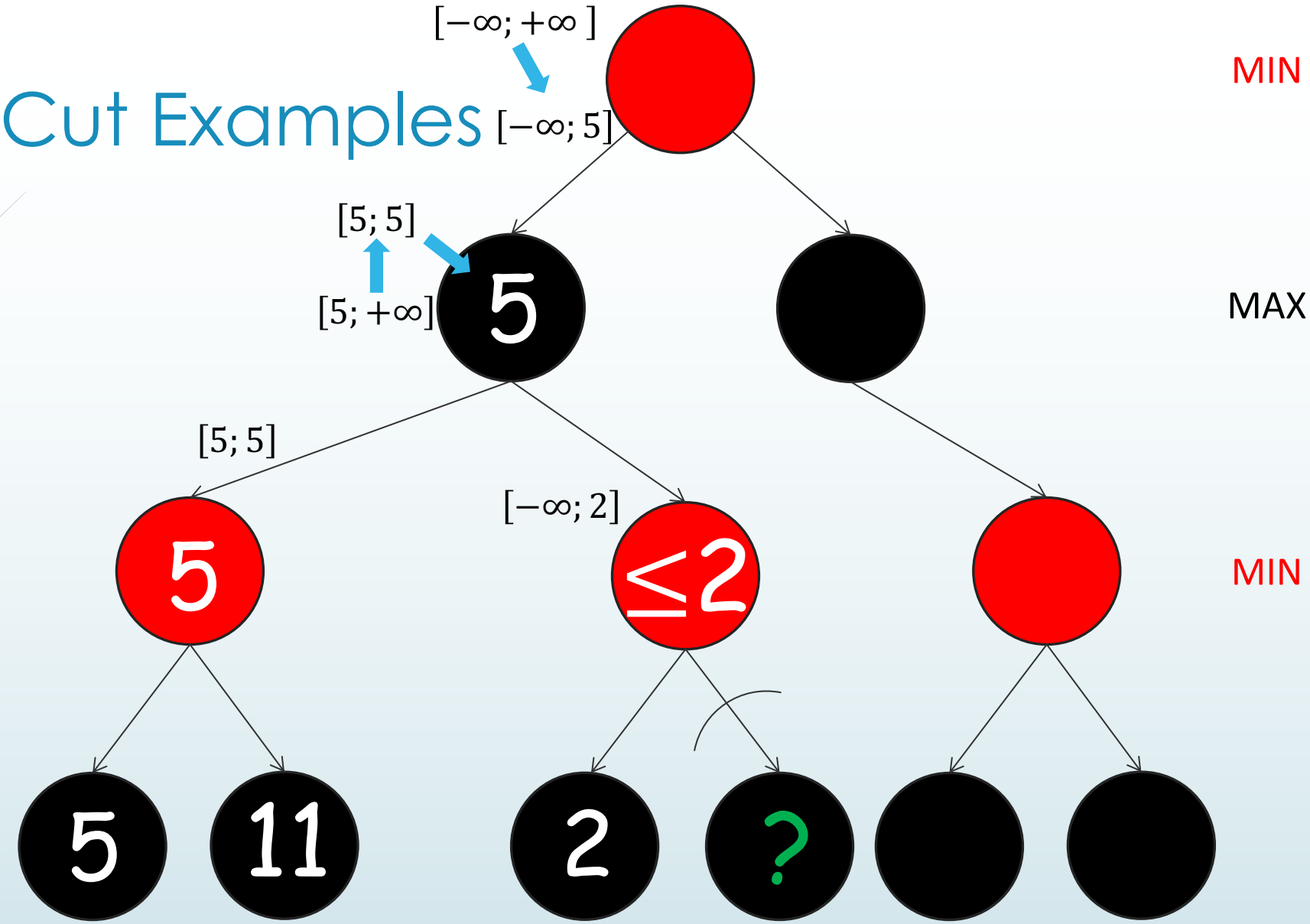
Cut Examples



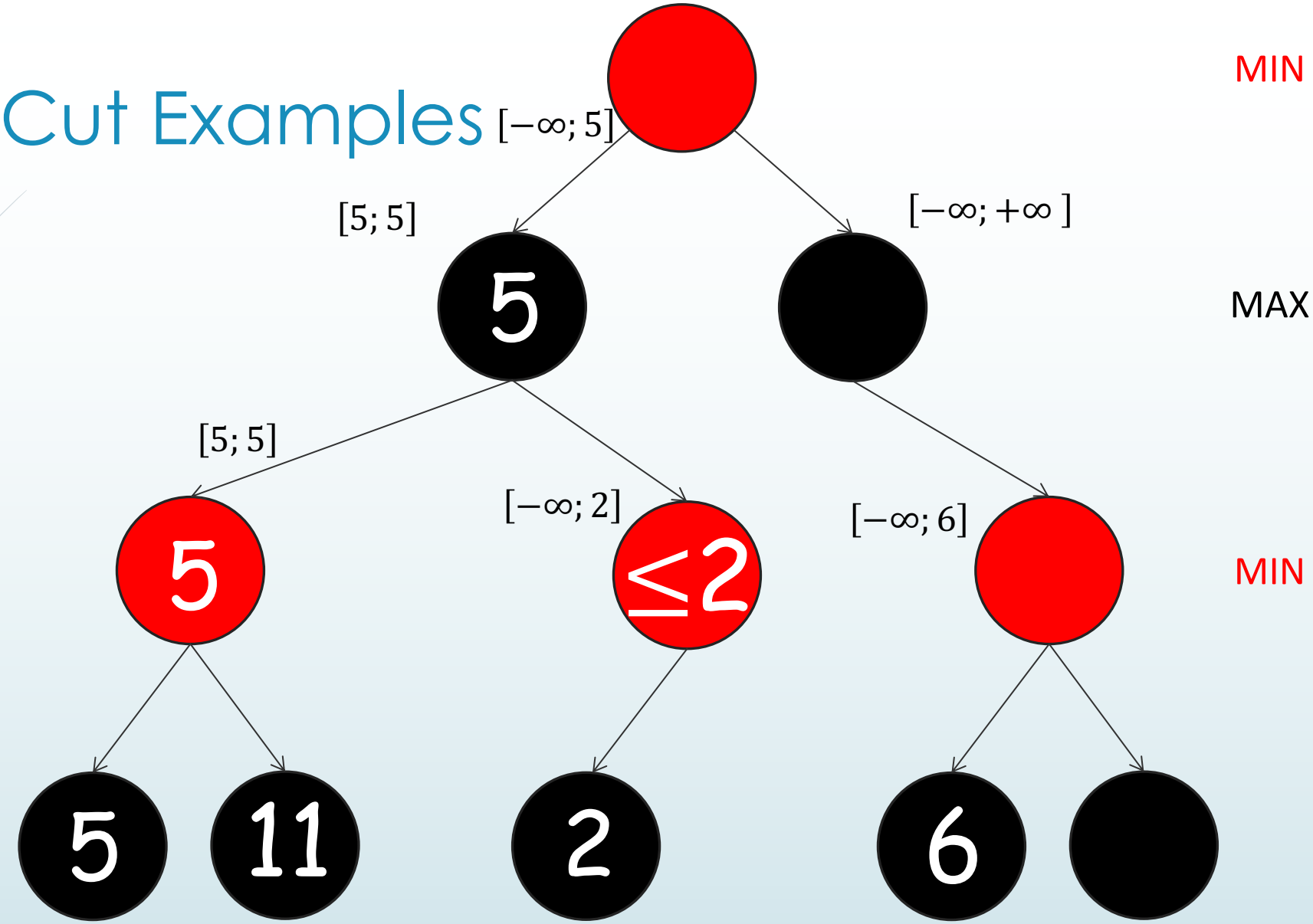
Cut Examples



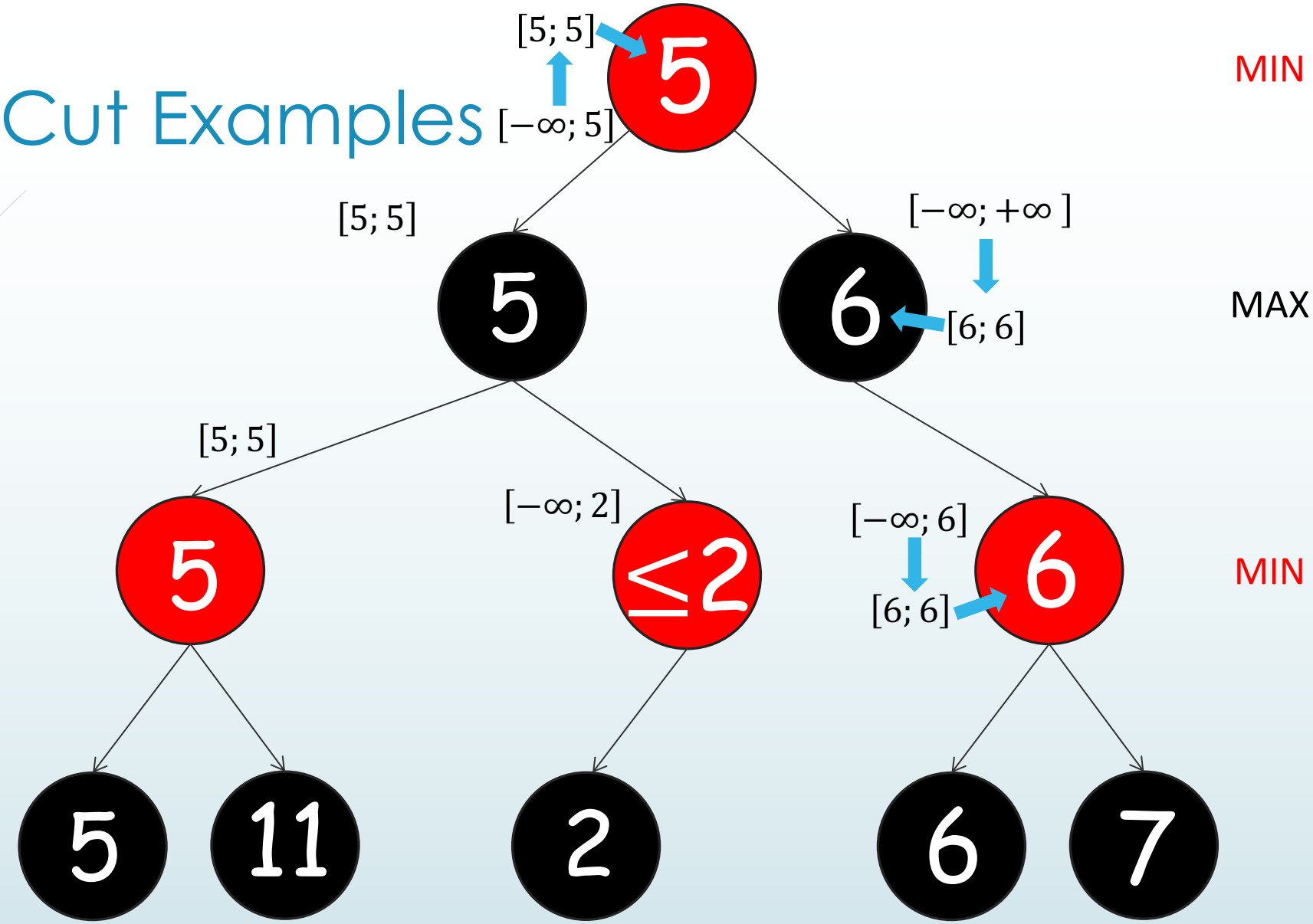
Cut Examples



Cut Examples



Cut Examples

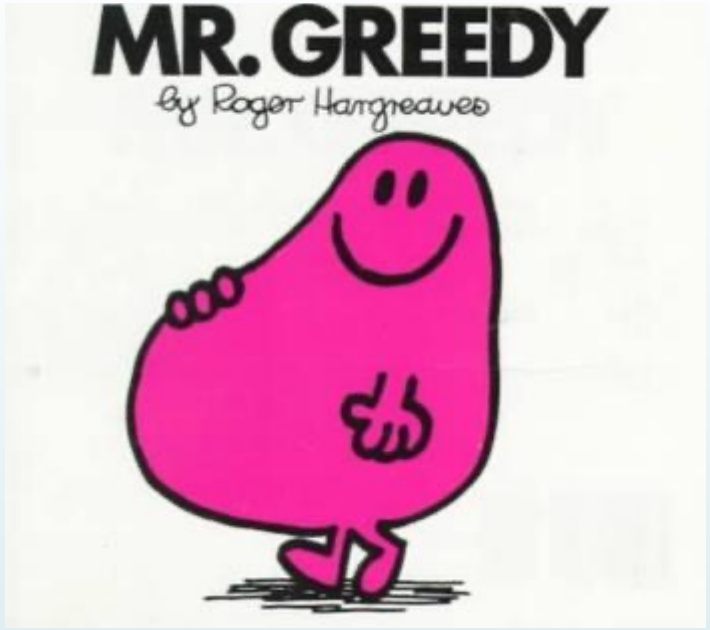


Alpha-Beta Pruning Problems

- The effectiveness of alpha–beta pruning is highly dependent on the order in which the states are examined
- Even with alpha–beta pruning and clever move ordering, minimax won't work for games like Chess and Go, because there are still too many states to explore in the time available
  - Too high *branching factor*
  - The branching factor is the average number of child nodes or potential moves/decisions at each node in a tree or graph, representing the "width" of the search space
    - Tic-Tac-Toe ~4
    - Chess ~31-35
    - Go ~250

Greedy Approaches

- Generate and Test Methods primarily
- Current best solutions are held
- When I find something better – I get rid of what I had
- Always wanting more
- Always takes the Locally Optimal choice

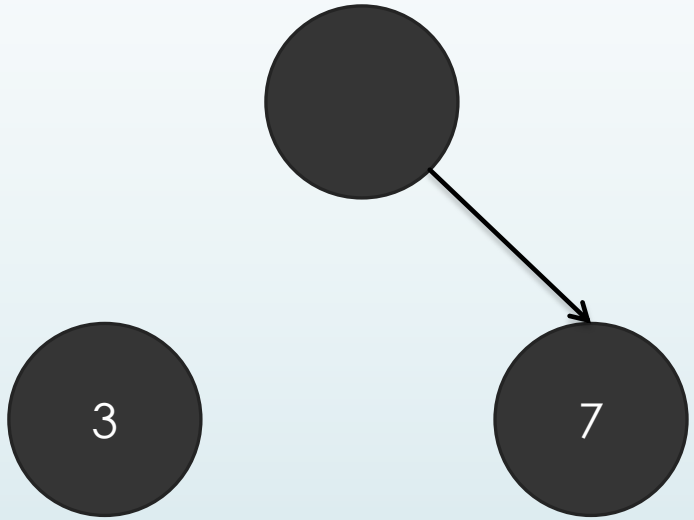


Greedy Algorithm Basics

- Start with a search space from which a candidate solution will be taken from
- There is a number of decisions to be made in sequence to find a solution, at each step make a choice which maximizes the outcome

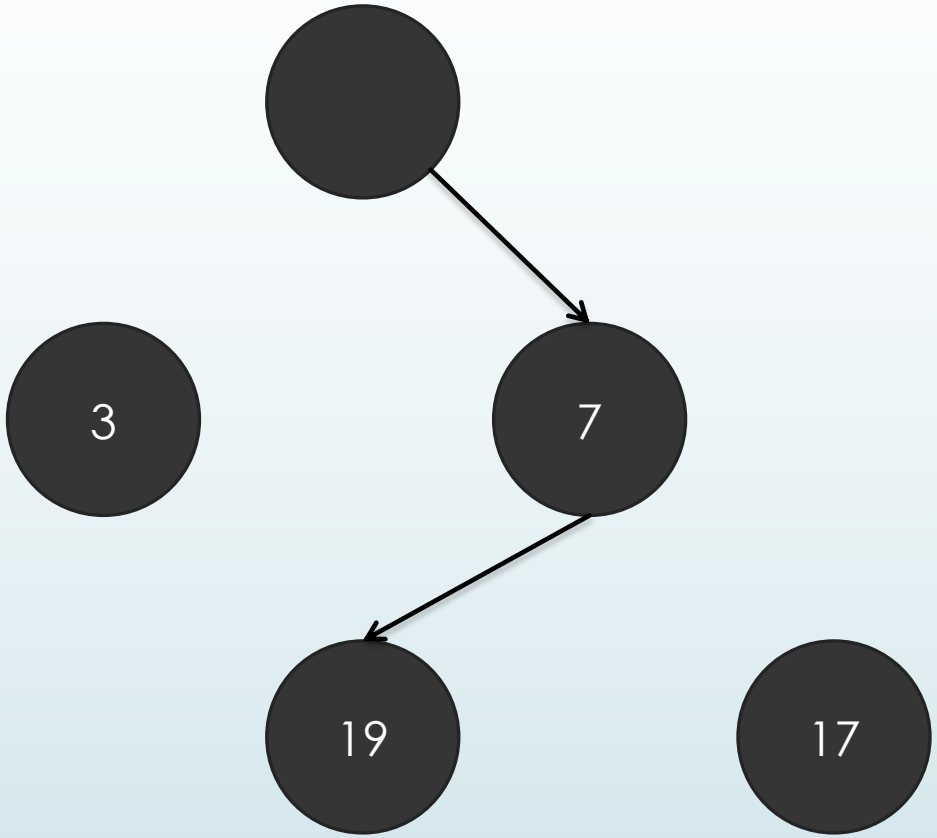
Example – Game Tree

- Two players are making moves in a game, each move gives a certain number of points to the player – goal is to make the most points



- In the first move our greedy player will take 7 points

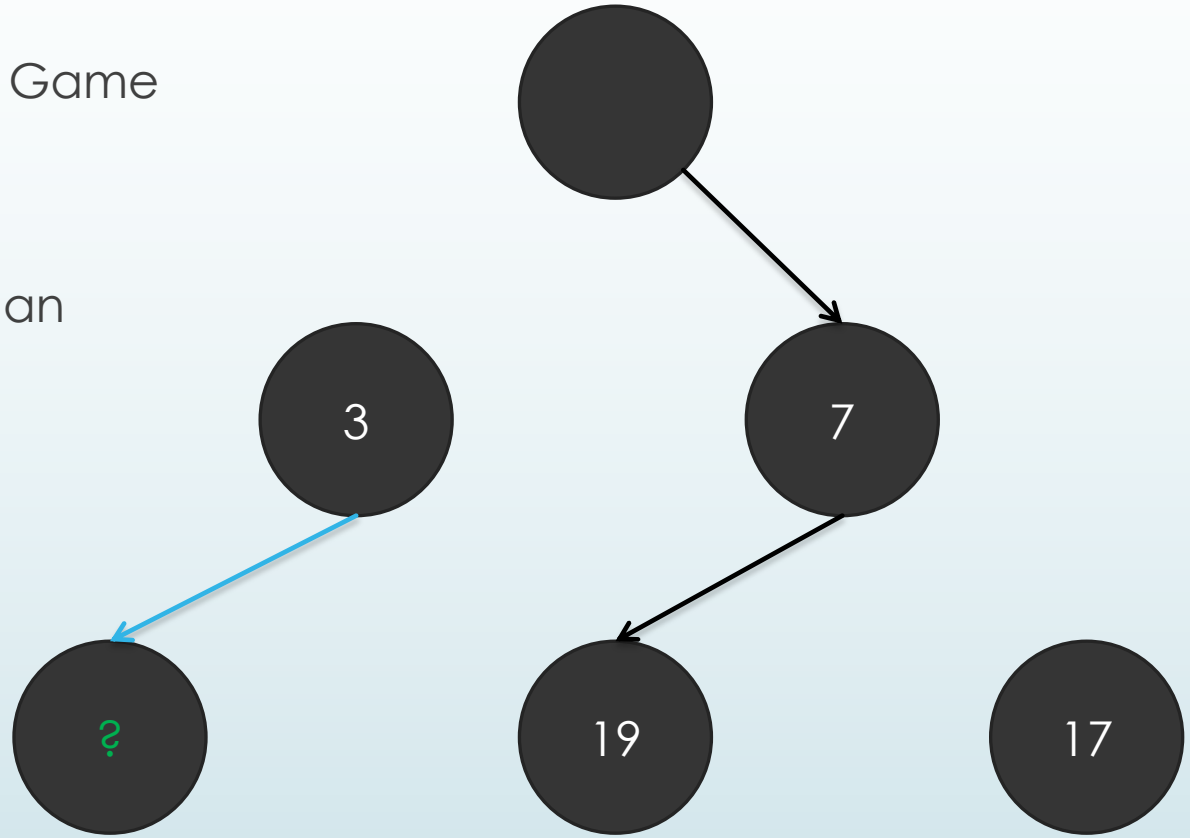
Example – Game Tree



- In the second move move our greedy player will take 19 points, for a total of 26 points

Was Greedy Good?

- Maybe – Depends on the Game being played
- If the ? Position was less than or equal to 23, then we have maximized the score
- If ? was greater, then we have failed

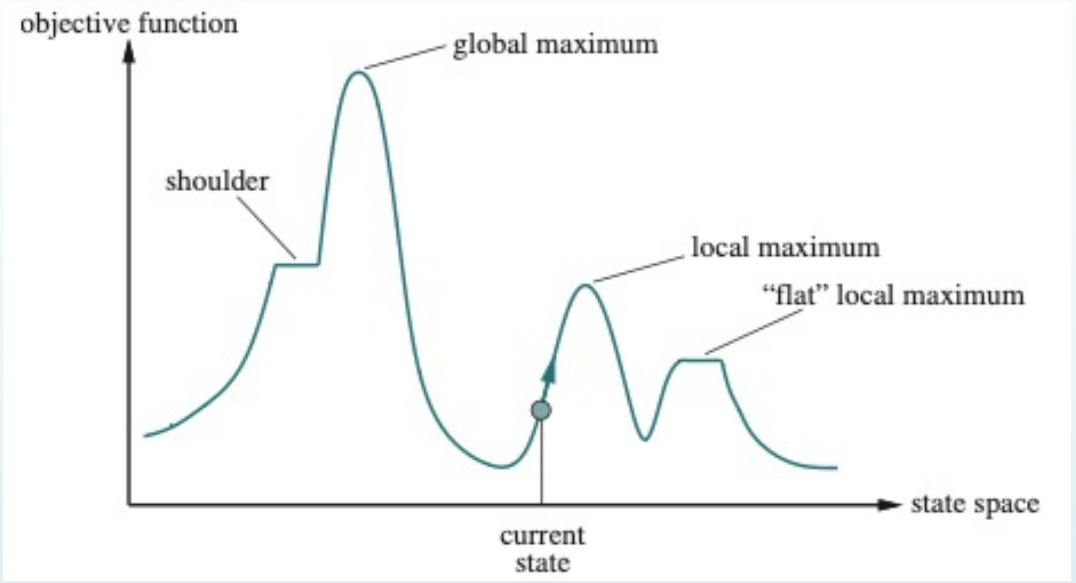


What Ensures That Greed Will Be Rewarded?

- Sub-problem optimization
  - Also called optimal substructures
  - The optimal solution to the problem contains optimal sub-problems
- Greedy Choice Property
  - We never need to consider the play until now and can move from this point only making the current best decision. We never need to review the past moves in order to make a better decision
  - In our example, if we have played the optimal game until now, then this starting point is also part of the optimal value on this move
- But the greediness sometimes does not work

Hill Climbers

- Single Searching point
- Search locally for changes to the actions
- Have a representation of a solution string
- Have a test of the representation about the space
- Each solution string maps to an outcome
  - An objective function
  - A fitness on the problem



Climbing Hill in Our Example

- Representation
  - String of actions which we can take
- Objective
  - Maximization of profit value based on the actions
  - No string of actions is allowed to have an overlap
- Local Change
  - Add and/or remove some number of items from the string
- Climbing the Hill
  - Replace my position if the change is better

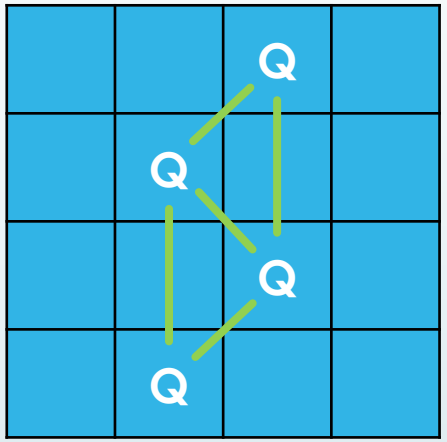
Hill-Climbing Algorithm

```
function HILL-CLIMBING(problem) returns a solution state
   inputs: problem, a problem
   static: current, a node
           next, a node

   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      next ← a highest-valued successor of current
      if VALUE[next] < VALUE[current] then return current
      current ← next
   end
```

Hill Climber for N Queens Problem

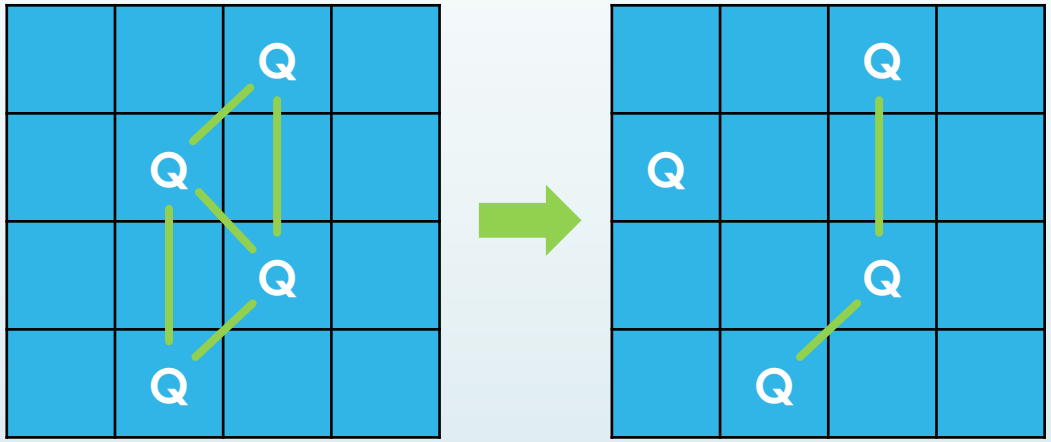
- h – number of queen pairs attacking each other
- $h \rightarrow min$



$$h = 5$$

Hill Climber for N Queens Problem

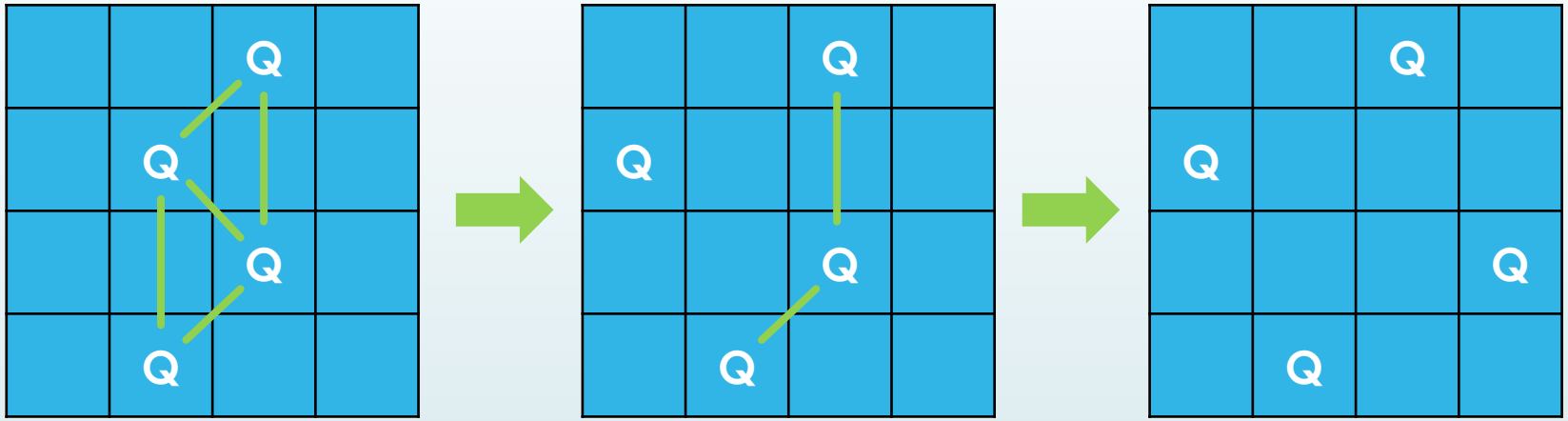
- h – number of queen pairs attacking each other
- $h \rightarrow min$



$h = 5$ $h = 2$

Hill Climber for N Queens Problem

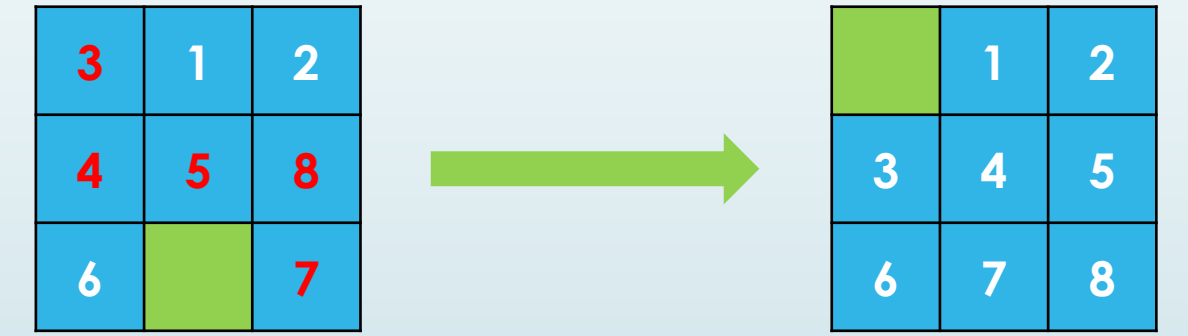
- h – number of queen pairs attacking each other
- $h \rightarrow min$



$h = 5$ $h = 2$ $h = 0$

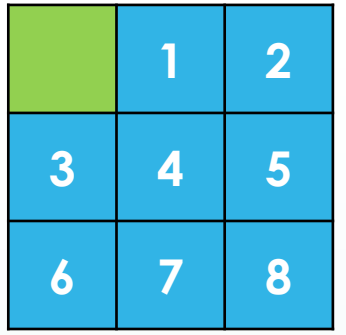
8-puzzle Problem

- The 8-puzzle problem is a puzzle invented and popularized by Noyes Palmer Chapman in the 1870s. It is played on a 3-by-3 grid with 8 square blocks labeled 1 through 8 and a blank square
- Goal is to rearrange the blocks so that they are in order. It is permitted to slide blocks horizontally or vertically into the blank square
- In general, there are 9!/2 = 181,400 reachable states



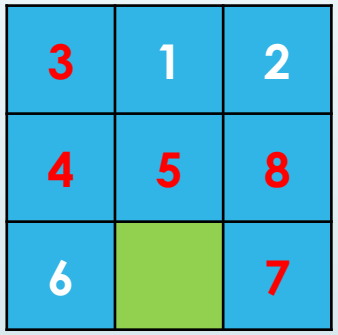
Initial state Goal state

Hill Climbers on 8-puzzle Problem



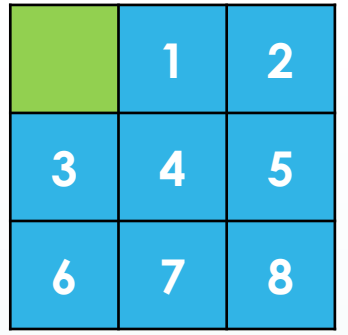
Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$



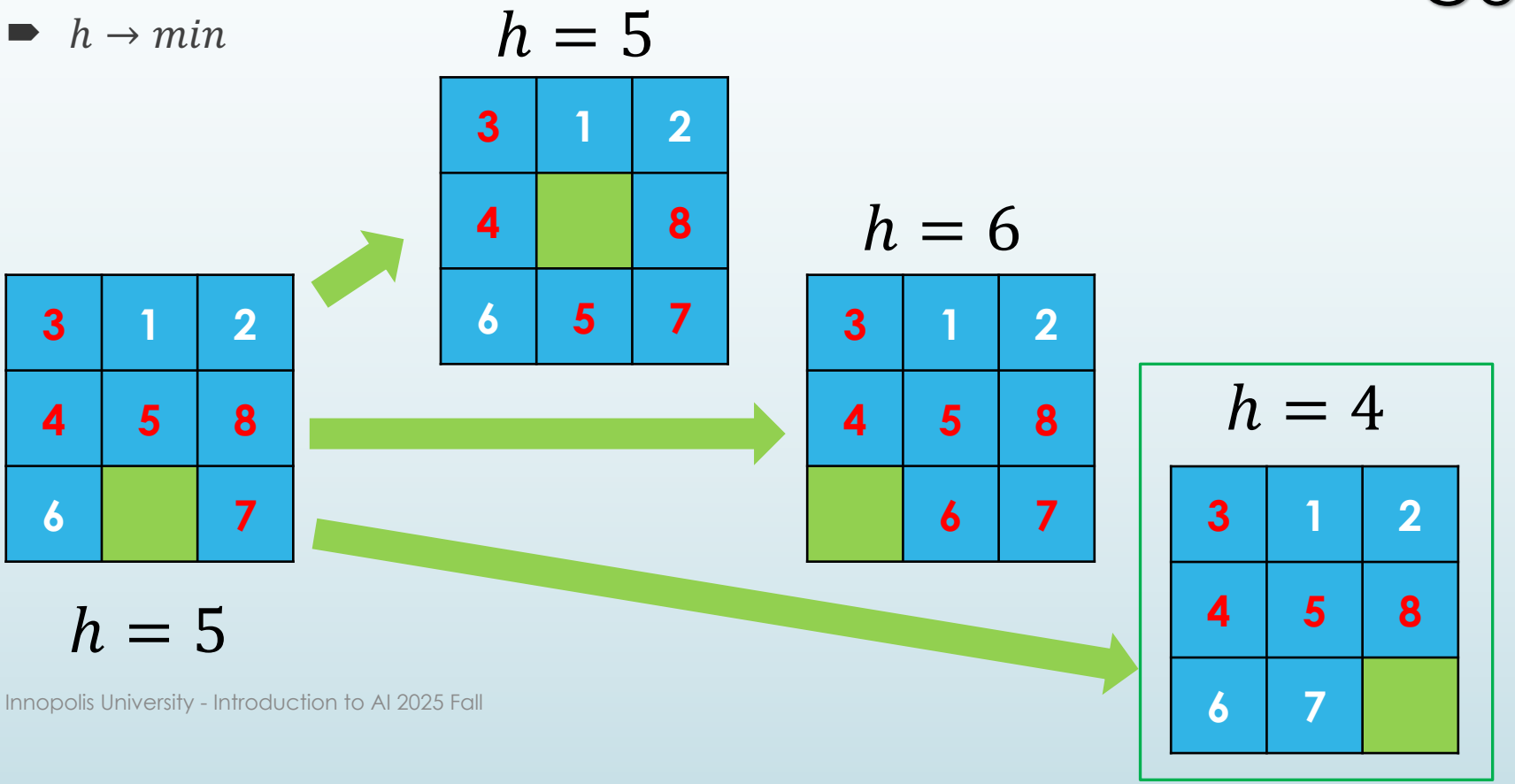
$h = 5$

Hill Climbers on 8-puzzle Problem

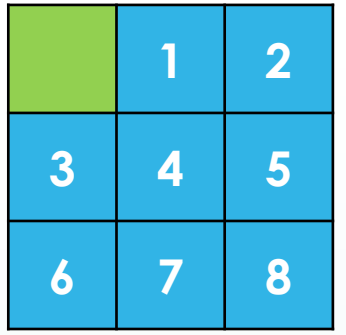


Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$

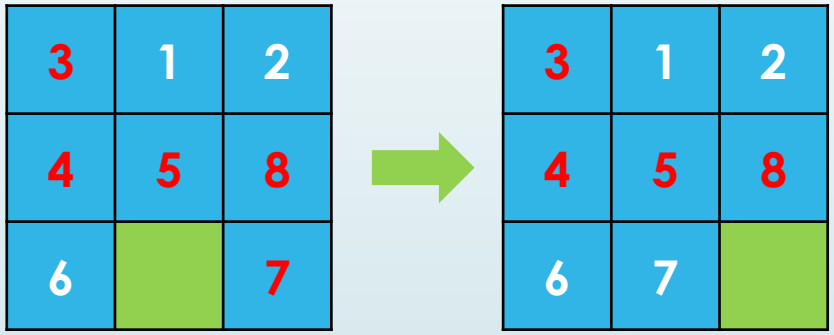


Hill Climbers on 8-puzzle Problem



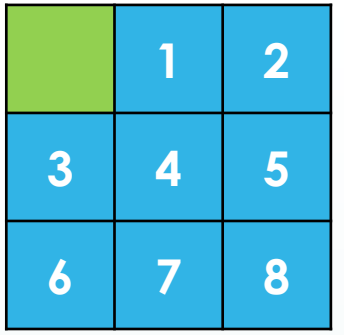
Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$



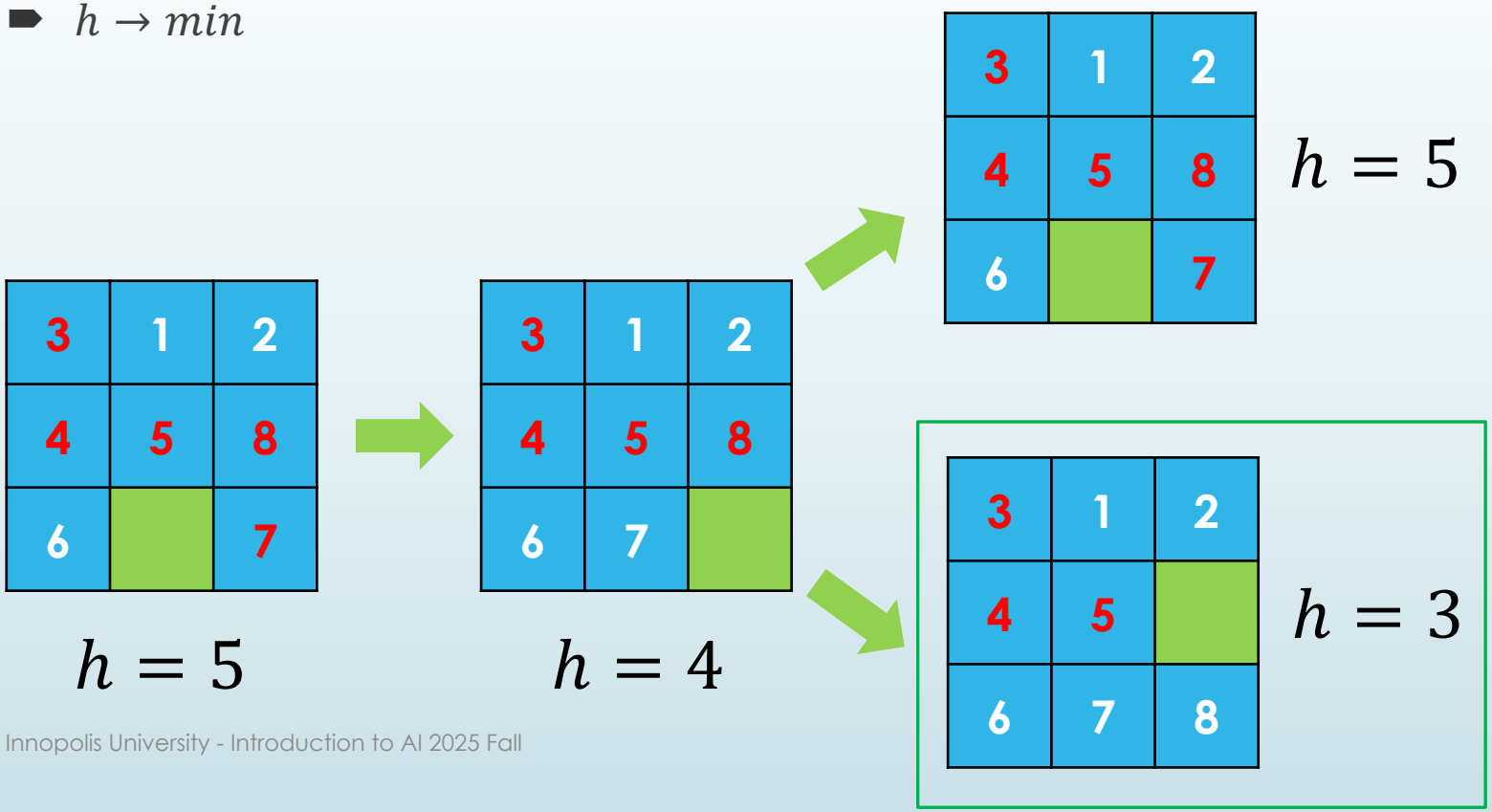
$h = 5$ $h = 4$

Hill Climbers on 8-puzzle Problem

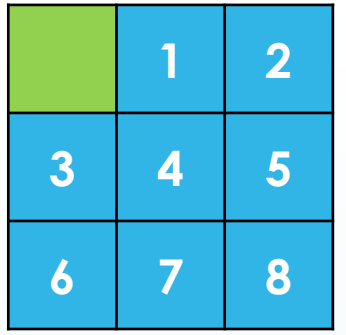


Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$

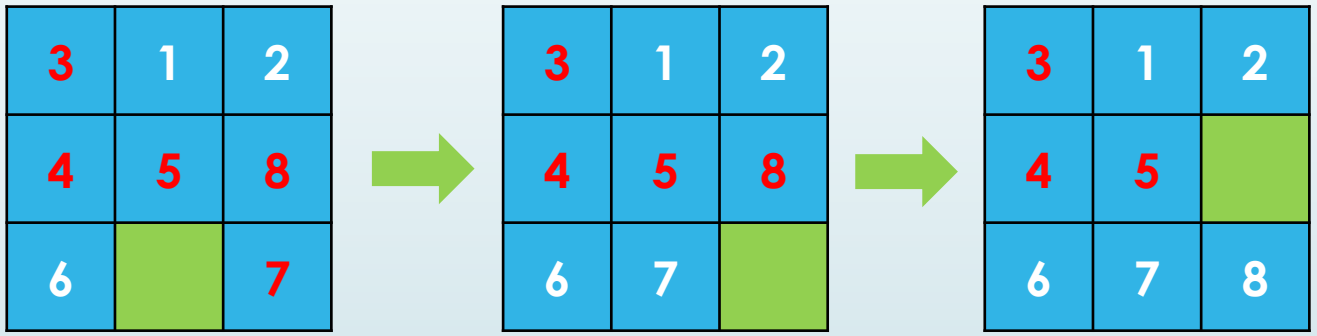


Hill Climbers on 8-puzzle Problem



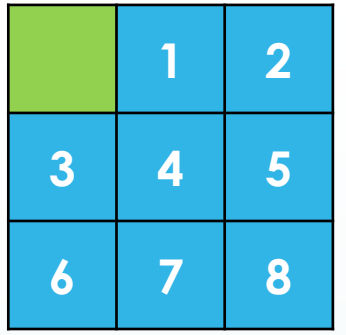
Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$



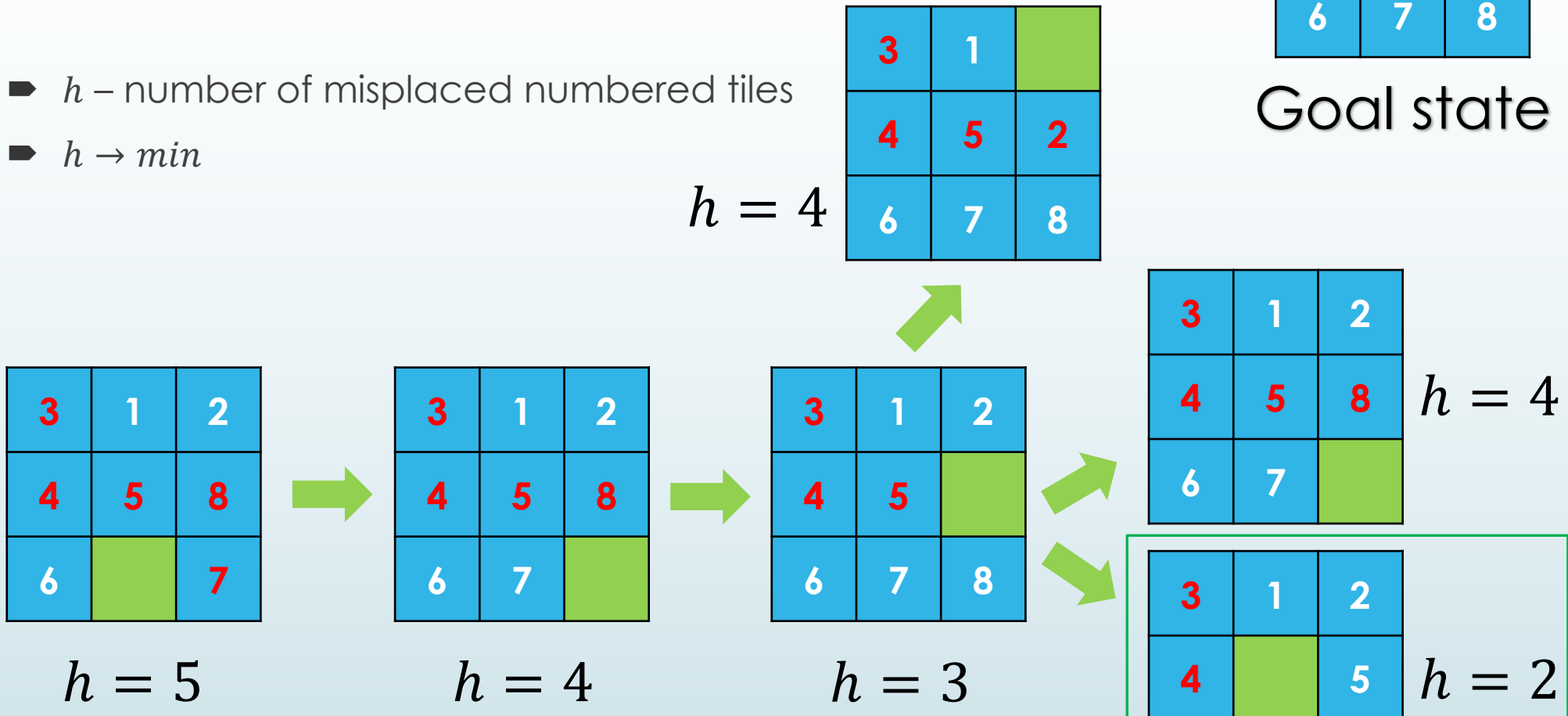
$h = 5$ $h = 4$ $h = 3$

Hill Climbers on 8-puzzle Problem

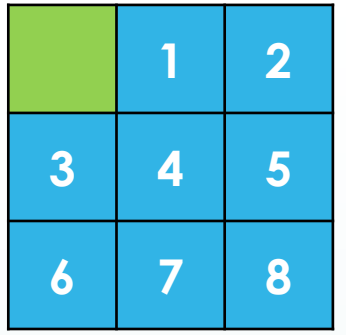


Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$

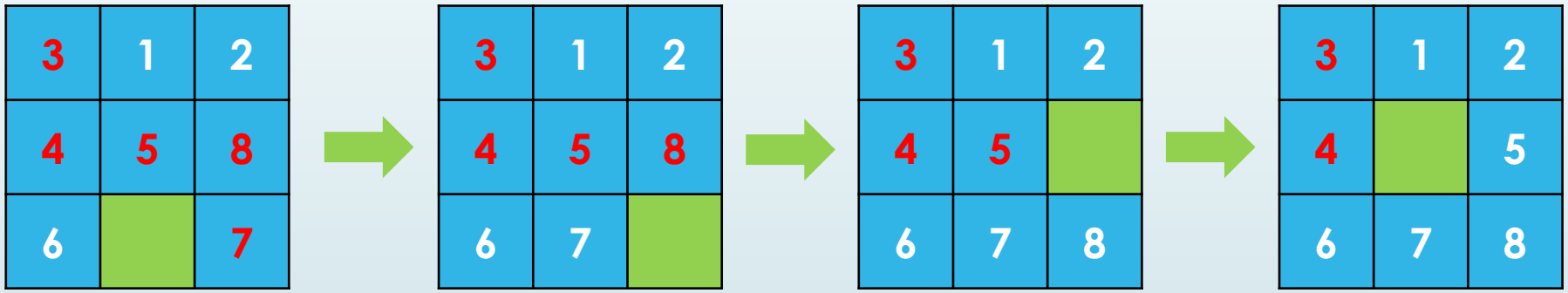


Hill Climbers on 8-puzzle Problem

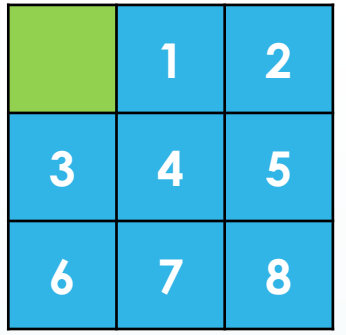


Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$

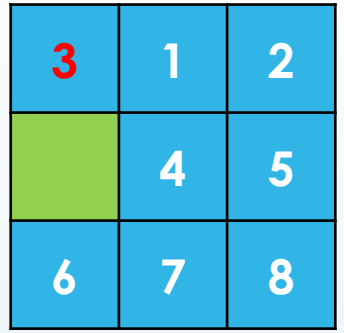


Hill Climbers on 8-puzzle Problem

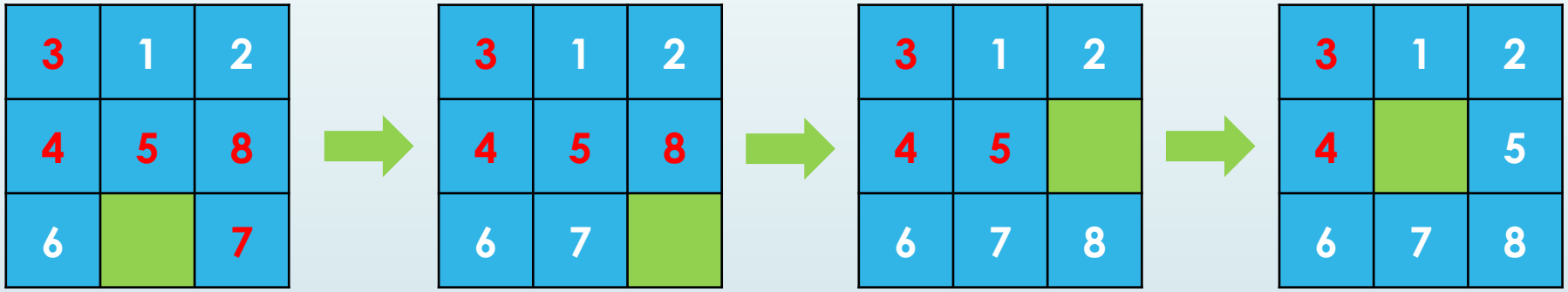


Goal state

- h – number of misplaced numbered tiles
- $h \rightarrow min$



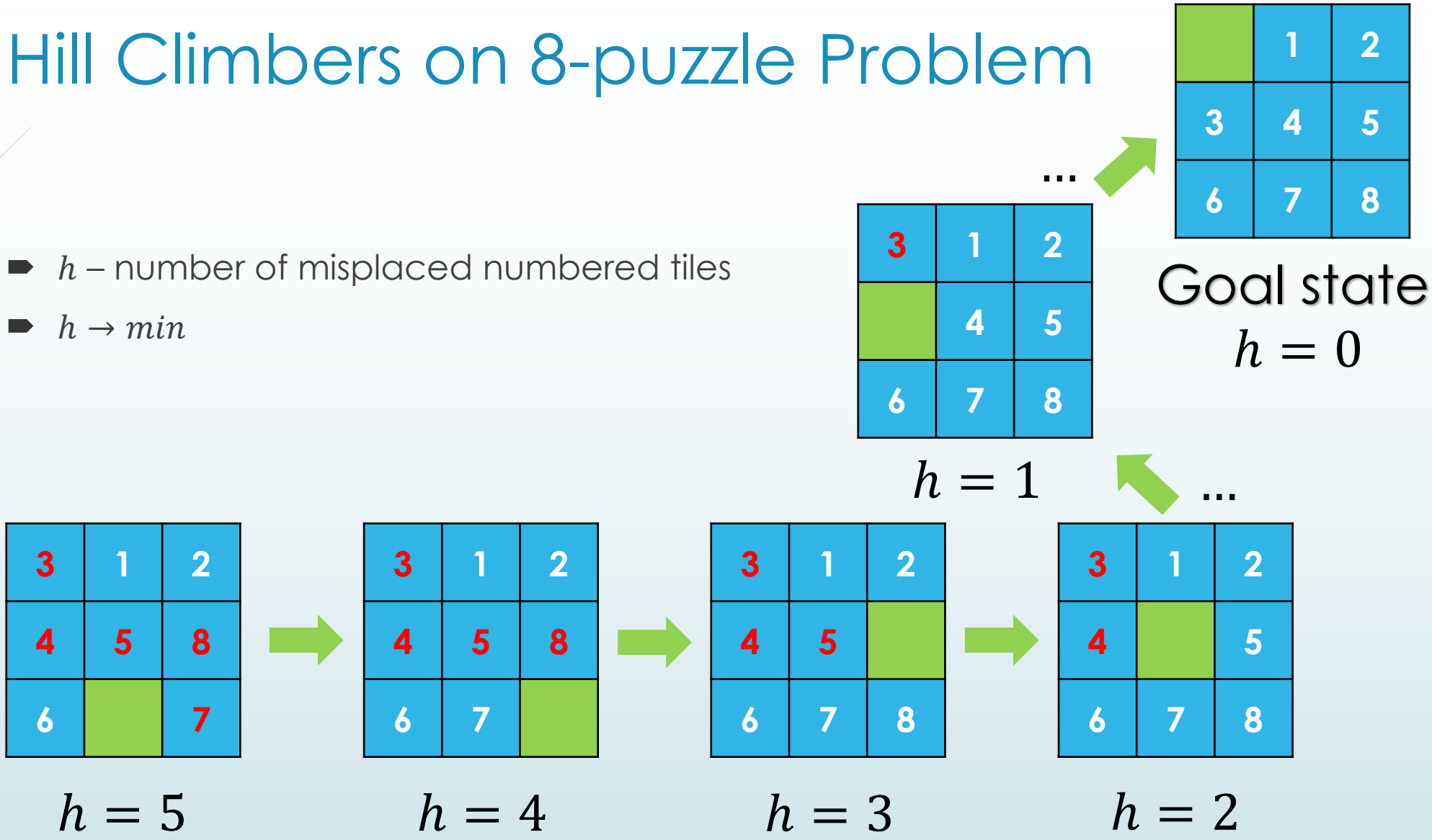
$h = 1$ ...



$h = 5$ $h = 4$ $h = 3$ $h = 2$

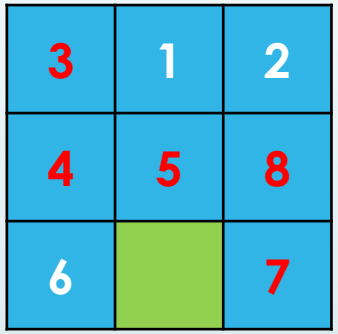
Hill Climbers on 8-puzzle Problem

- h – number of misplaced numbered tiles
- $h \rightarrow min$

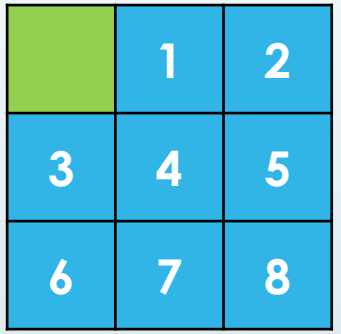


Alternative Heuristics?

- Instead of number of misplaced tiles we could use the sum of the *Manhattan distances* of the tiles from their goal positions as heuristics
- $h \rightarrow min$
- Test it by yourself



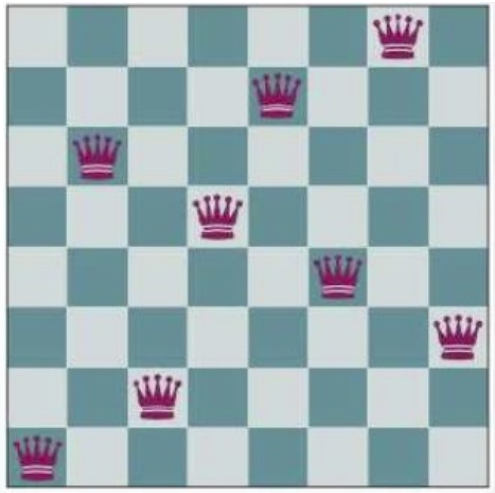




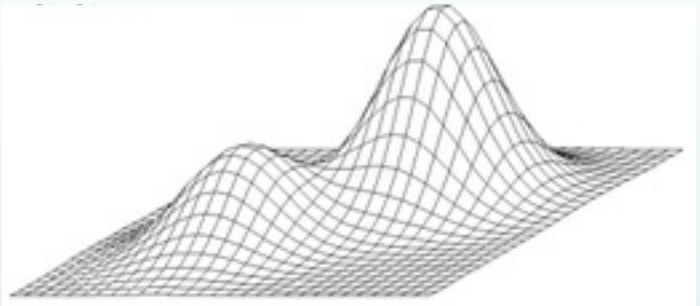
$$h = 0 + 0 + 1 + 1 + 1 + 0 + 1 + 1 = 5$$

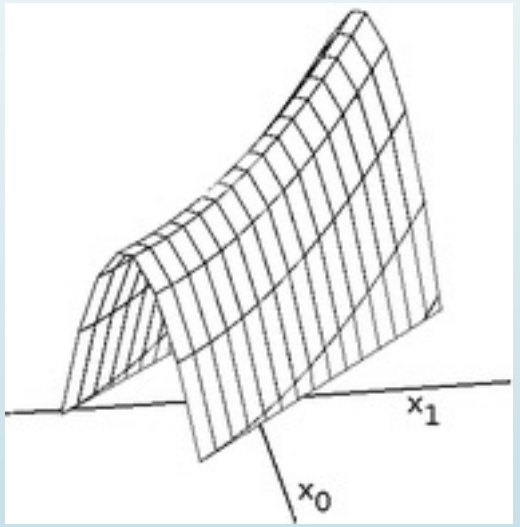
Goal state

Issues and Work About



- Local Maxima
  - Stuck in suboptimal places if our value is on a localized hill
    - Change the mutation
    - Change the representation
    - Random restart
- Plateaus
  - A flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists
- Ridges
  - Because hill climbers only adjust one element in the vector at a time, each step will move in an axis-aligned direction





Issues and Work About

- Multiple Optima
  - Multiple Good Solutions
    - Random Restart
  - Large areas of equal solutions
    - Change the mutation
    - Random Restart
- Jumping over areas of the space
  - Selected Mutation hops over the space

Summary

- Uniformed Search
- Informed Search
- Backtracking
- Hill Climbers

Next week: Learning by Searching (Part II)

References

- Minimax with alpha-beta pruning
- One more minimax with alpha-beta pruning
- Hill Climbers